yokshita@yokshithachowdary

GNU nano 8.7.1

```c
#include <stdio.h>

int main()
{
    int a[6][6];
    int n, t;
    int i, j;
    int sum, max;
    int rowMax, colMax;
    int mainDia = 0, secDia = 0;
    int boundary = 0;
    int count = 0, hotspots = 0;

    printf("Enter grid size (3 to 6): ");
    scanf("%d", &n);

    while (n < 3 || n > 6)
    {
        printf("Invalid size. Enter again: ");
        scanf("%d", &n);
    }

    printf("Enter risk scores (0 to 100):\n");

    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &a[i][j]);
```

^G Help ^O Write Out ^F Where Is ^K Cut
^X Exit ^R Read File ^\ Replace ^U Paste

26°C
Mostly clear
Search

yokshita@yokshithachowdary

GNU nano 8.7.1

```
            while (a[i][j] < 0 || a[i][j] > 100)
            {
                printf("Invalid value. Enter again: ");
                scanf("%d", &a[i][j]);
            }
        }
    }

    printf("Enter warning threshold (1 to 100): ");
    scanf("%d", &t);

    while (t < 1 || t > 100)
    {
        printf("Invalid threshold. Enter again: ");
        scanf("%d", &t);
    }

    // Row traversal
    rowMax = 0;

    for (i = 0; i < n; i++)
    {
        sum = 0;

        for (j = 0; j < n; j++)
            sum += a[i][j];

        if (sum > rowMax)
            rowMax = sum;
```

^G Help ^O Write Out ^F Where Is ^K Cut
^X Exit ^R Read File ^\ Replace ^U Paste

26°C Mostly clear

Search

yokshita@yokshithachowdary

GNU nano 8.7.1

```
    }

    printf("\nRow Totals:\n");

    for (i = 0; i < n; i++)
    {
        sum = 0;

        for (j = 0; j < n; j++)
            sum += a[i][j];

        printf("Row %d = %d\n", i + 1, sum);
    }

    printf("Highest row(s): ");

    for (i = 0; i < n; i++)
    {
        sum = 0;

        for (j = 0; j < n; j++)
            sum += a[i][j];

        if (sum == rowMax)
            printf("%d ", i + 1);
    }

    // Column traversal
    colMax = 0;
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T
^X Exit ^R Read File ^\ Replace ^U Paste ^J

26°C
Mostly clear
Search

yokshita@yokshithachowdary

GNU nano 8.7.1

```
    for (j = 0; j < n; j++)
    {
        sum = 0;

        for (i = 0; i < n; i++)
            sum += a[i][j];

        if (sum > colMax)
            colMax = sum;
    }

    printf("\n\nColumn Totals:\n");

    for (j = 0; j < n; j++)
    {
        sum = 0;

        for (i = 0; i < n; i++)
            sum += a[i][j];

        printf("Column %d = %d\n", j + 1, sum);
    }

    printf("Highest column(s): ");

    for (j = 0; j < n; j++)
    {
        sum = 0;
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T
^X Exit ^R Read File ^\ Replace ^U Paste ^J

26°C Mostly clear

Search

yokshita@yokshithachowdary

GNU nano 8.7.1

```
        for (i = 0; i < n; i++)
            sum += a[i][j];

        if (sum == colMax)
            printf("%d ", j + 1);
    }

    // Full grid traversal
    max = a[0][0];

    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            if (a[i][j] > max)
                max = a[i][j];

            if (a[i][j] >= t)
                count++;
        }
    }

    printf("\n\nMaximum risk score = %d\n", max);
    printf("Maximum score positions:\n");

    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T
^X Exit ^R Read File ^\ Replace ^U Paste ^J

26°C Mostly clear

Search

```
        for (j = 0; j < n; j++)
        {
            if (a[i][j] == max)
                printf("(%d, %d) ", i + 1, j + 1);
        }
    }

    printf("\nCells >= threshold = %d\n", count);

    // Diagonal traversal
    for (i = 0; i < n; i++)
    {
        mainDia += a[i][i];
        secDia += a[i][n - 1 - i];
    }

    printf("\nMain diagonal sum = %d", mainDia);
    printf("\nSecondary diagonal sum = %d", secDia);

    if (mainDia == secDia)
        printf("\nDiagonals are BALANCED\n");
    else
        printf("\nDiagonals are NOT BALANCED\n");

    // Boundary traversal
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
```


26°C
Mostly clear
Search

yokshita@yokshithachowdary

GNU nano 8.7.1

```
            if (i == 0 || i == n - 1 ||
                j == 0 || j == n - 1)
            {
                boundary += a[i][j];
            }
        }
    }

    printf("\nBoundary total = %d\n", boundary);

    // Local hotspot detection
    for (i = 1; i < n - 1; i++)
    {
        for (j = 1; j < n - 1; j++)
        {
            if (a[i][j] >= t &&
                a[i][j] > a[i - 1][j] &&
                a[i][j] > a[i + 1][j] &&
                a[i][j] > a[i][j - 1] &&
                a[i][j] > a[i][j + 1])
            {
                printf("Hotspot at (%d, %d)\n",
                       i + 1, j + 1);

                hotspots++;
            }
        }
    }
|
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Exe
^X Exit ^R Read File ^\ Replace ^U Paste ^J Just

26°C Mostly clear

Search

yokshita@yokshithachowdary

GNU nano 8.7.1

```
            if (a[i][j] >= t &&
                a[i][j] > a[i - 1][j] &&
                a[i][j] > a[i + 1][j] &&
                a[i][j] > a[i][j - 1] &&
                a[i][j] > a[i][j + 1])
            {
                printf("Hotspot at (%d, %d)\n",
                       i + 1, j + 1);

                hotspots++;
            }
        }
    }

    printf("Total hotspots = %d\n", hotspots);

    // Final decision
    printf("\nSecurity Status: ");

    if (hotspots > 0)
        printf("CRITICAL\n");
    else if (count >= n)
        printf("WARNING\n");
    else
        printf("NORMAL\n");

    return 0;
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T
^X Exit ^R Read File ^\ Replace ^U Paste ^J

26°C Mostly clear

Search

yokshita@yokshithachowdary

```
Enter grid size (3 to 6): 3
Enter risk scores (0 to 100):
10 20 30
40
90 60
70 80 50
Enter warning threshold (1 to 100): 60

Row Totals:
Row 1 = 60
Row 2 = 190
Row 3 = 200
Highest row(s): 3

Column Totals:
Column 1 = 120
Column 2 = 190
Column 3 = 140
Highest column(s): 2

Maximum risk score = 90
Maximum score positions:
(2, 2)
Cells >= threshold = 4

Main diagonal sum = 150
Secondary diagonal sum = 190
Diagonals are NOT BALANCED

Boundary total = 360
Hotspot at (2, 2)
Total hotspots = 1
```

26°C
Mostly clear
Search

yokshita@yokshithachowdary

```
Total hotspots = 1

Security Status: CRITICAL
yokshita@yokshithachowdary:~$ ./a.out
Enter grid size (3 to 6): 4
Enter risk scores (0 to 100):
10 20 30 40
20 90 50 70
33 70 68 75
90 69 45 30
Enter warning threshold (1 to 100): 50

Row Totals:
Row 1 = 100
Row 2 = 230
Row 3 = 246
Row 4 = 234
Highest row(s): 3

Column Totals:
Column 1 = 153
Column 2 = 249
Column 3 = 193
Column 4 = 215
Highest column(s): 2

Maximum risk score = 90
Maximum score positions:
(2, 2) (4, 1)
Cells >= threshold = 8

Main diagonal sum = 198
Secondary diagonal sum = 250
Diagonals are NOT BALANCED

Boundary total = 532
Hotspot at (2, 2)
Total hotspots = 1

Security Status: CRITICAL
yokshita@yokshithachowdary:~$ nano p8.c
yokshita@yokshithachowdary:~$
```